

Explaining the Behavior of an Instruction-Driven Research Agent: A Pilot Study

Christian Block

July 2026

Abstract

A language model can be set up to run a small research loop on its own: given a text file of instructions and a way to run experiments, it repeatedly proposes something to try, runs it, looks at the result, and decides what to do next. Change the instructions and the loop behaves differently, but it is usually unclear which part of the instructions is responsible for which part of the difference. This report treats the instruction file as a small configuration, in the same way an algorithm's hyperparameters form a configuration, and borrows a standard technique from automated machine learning – ablation studies over a fractional factorial design – to measure the effect of each instructional choice on the loop's behavior. Five instructional choices were varied (how precisely the evaluation metric is defined, whether the agent may compare two model families or must stick to one, whether its experiment budget is fixed or adaptive, whether it must explain its reasoning, and whether it favors broad exploration or immediate refinement), an eight-configuration ablation design was run against a real machine learning task (tuning Random Forest and Gradient Boosting classifiers), and a simple statistical model was fit to explain which instructional choice drives which aspect of the resulting behavior. The method correctly recovers an expected relationship (the instruction to write full explanations is, unsurprisingly, the dominant driver of how much text the agent produces) and also finds a less obvious one: whether the agent is told to explore broadly or to refine immediately turns out to be the strongest single predictor of the accuracy it eventually reaches. A held-out prediction, made before the corresponding configuration was ever run, is checked against a real execution: one quantity is predicted almost exactly, another is off by a small but informative amount. A sixth quantity, how closely each run's sequence of actions resembles a reference research session, is also measured and modeled, using a reference trace produced for this pilot in place of an actual recorded human session (see below). All numbers in this report come from real, executed scikit-learn experiments; nothing is simulated. The report closes with an honest account of what this pilot could not do – in particular, that the language-model role in every run, and the reference research session it is compared against, were both played by the author rather than by an independent model call or an actual recorded person – and what a larger follow-up would need to change.

1 Introduction

Suppose you want a language model – a program trained on large amounts of text that, given a prompt, produces further text – to act as a small autonomous researcher. You give it a description of a task, for example "find a good Random Forest configuration for this dataset", together with a tool it can call to actually run an experiment and get a number back. The model reads the instructions, proposes something to try, calls the tool, reads the result, and decides what to try next, repeating this until it decides to stop. Systems built this way are sometimes called autoresearch loops, and they are attractive precisely because they need so little supervision: point one at a problem, and it works through the same cycle a human researcher would – hypothesize, run an experiment, interpret, decide – without a person in the loop.

The entire behavior of such a system, however, is steered by one artifact: the instruction file that starts it off. We refer to this file throughout as `program.md`. Anyone who has actually run one of these

loops more than a few times notices something quickly: small changes to the wording of the instructions produce noticeably different research behavior. Tell the agent to stop after a fixed number of experiments and it stops after a fixed number of experiments; phrase the stopping rule differently and it might run twice as many, or half as many. Ask for a precise evaluation metric and it reasons about numbers differently than if the metric is left vague. This is not surprising in itself. What is unsatisfying is that the relationship is usually left as an observation rather than an explanation: we notice that the instructions matter, but we do not have a way of saying which specific part of the instructions is responsible for which specific part of the resulting behavior, nor by how much.

This gap matters because it prevents two things a scientist normally wants from an experimental result. First, it prevents prediction: without knowing which instructional choice drives which behavior, we cannot say in advance what will happen if we try a combination of instructions we have not yet tried. Second, it prevents explanation to someone else: "the agent behaved differently because I changed the instructions" is not a statement anyone else can act on, whereas "the agent ran fewer experiments because the stopping rule was stricter, and this accounts for about half of the variation we saw" is.

This project treats that gap as a measurement problem rather than a mystery, and borrows the tools for it from a field that already solved a structurally identical problem: automated machine learning. When a learning algorithm has several hyperparameters, researchers do not simply observe that changing hyperparameters changes performance and leave it there. They routinely apply techniques – ablation analysis, fractional factorial experiment design, and variance decomposition over hyperparameters – to work out which hyperparameters matter, how much each one matters, and whether they interact [1, 2, 3]. Our central observation is that an instruction file like `program.md` is, structurally, no different from a hyperparameter vector: it is a set of discrete, largely independent choices that jointly determine a system's behavior. If that observation is right, the same tools should transfer directly, and that transfer is what this report tries out.

To keep the experiment small enough to actually run and check by hand, the domain was restricted to a single, well-understood machine learning task: tuning Random Forest and Gradient Boosting classifiers on one fixed, small benchmark dataset. Nothing about this report is trying to show that these particular models can be tuned well – that is a solved problem. The point of choosing a simple, familiar task is that it lets us focus entirely on the question that actually matters here: given a fixed task, does changing the instructions change the research *process* in a way we can measure, attribute, and predict?

The rest of this report is organized as follows. Section 2 briefly explains the three ideas borrowed from automated machine learning, for readers who have not seen them before. Section 3 walks through the six steps of the research plan that was executed, one by one. Section 4 states the exact experimental setup. Section 5 presents and discusses the results. Section 7 is a frank account of what this pilot did not manage to do and why. Section 8 concludes.

2 Background

Three ideas underlie this project, all originally developed to make sense of algorithms with many hyperparameters rather than instruction files, but they transfer without modification.

The first is *variance decomposition over configuration*. If an algorithm has several hyperparameters and its performance varies as we change them, a natural question is how much of that variation each hyperparameter is individually responsible for. Functional ANOVA [2] answers this by treating the outcome (say, validation accuracy) as a function of the hyperparameter vector and decomposing its variance into a part attributable to each hyperparameter on its own, plus parts attributable to interactions between pairs of hyperparameters. The result is a short list of interpretable numbers – "hyperparameter A explains 60% of the variance, hyperparameter B explains 5%" – instead of a black-box performance curve. This report reuses exactly this idea, with axes of an instruction file standing in for hyperparameters.

The second is *ablation analysis*, which asks a related but distinct question: given two specific configurations with different performance, which of the individual differences between them is responsible for how much of the gap [1]. This complements variance decomposition by giving a step-by-step account of one specific comparison, rather than a global summary across many configurations.

The third is a practical constraint that both of the above run into: measuring the effect of k two-level choices in full requires testing all 2^k combinations, which becomes expensive quickly. *Fractional factorial design* sidesteps this by testing a carefully chosen subset of combinations, small enough to be affordable, while still guaranteeing that the individual ("main") effect of every choice can be estimated without bias from the others. The classical construction for this, due to Plackett and Burman [3], produces such a subset in multiples of four runs – eight runs suffice for up to seven two-level choices – and is standard practice in industrial experimentation. The one thing it gives up, in exchange for needing so few runs, is the ability to tell apart interaction effects between pairs of choices; we return to why this matters in Section 5.

Put together, these three ideas describe a small pipeline: pick a manageable subset of configurations to actually run (fractional factorial design), fit a simple model that attributes the resulting variation in behavior to individual configuration choices (variance decomposition), and use that model to reason about specific comparisons (ablation-style reasoning). This is exactly the pipeline applied below to `program.md`.

3 Method

The project follows a six-step plan. All six steps were carried out for this pilot, using real, executed machine learning experiments throughout, though Step 1 required a compromise explained below. The overall flow is: produce a reference research trace to compare everything else against (Step 1); turn the instruction file into a configuration (Step 2); decide which configurations to actually run (Step 3); run them against a real experiment executor (also part of Step 3, described together with the loop mechanics); turn each run's record into a handful of numbers (Step 4); fit a model relating configuration to those numbers (Step 5); and check that model's predictions against a real, held-out run (Step 6).

3.1 Step 1: a human reference trace

The original plan for this project called for a human researcher to work through one concrete instance of the same kind of problem the agent is given – for example, "does the feature-subsampling ratio affect generalization of a Random Forest on this dataset" – while recording their screen and thinking aloud, with the resulting recording transcribed and split into a sequence of steps using a small fixed vocabulary: hypothesize, design an experiment, run it, observe the result, interpret it, decide what to do next. Such a trace would give a genuine point of comparison: instead of only asking whether the agent's behavior changes with the instructions, one could also ask how close the agent's research process comes to how a person would actually go about the same problem.

That is not what happened here, and it is worth being precise about the difference. No person sat down and was recorded solving this problem. Instead, at the author's direction, the reference trace was authored directly: working through the feature-subsampling question above, deciding what to try at each step, and writing the resulting reasoning down in the first person, in the style of a think-aloud recording. The experiments described in that trace were actually run against the same executor used everywhere else in this project, so every number quoted in it is real. What is not real is the reasoning itself – it is a plausible reconstruction of how a person might approach the problem, not an observation of how one actually did. This is a materially weaker substitute than a genuine recording, and it is treated that way throughout this report: wherever the trace is used below, it is described as a reference trace, never as evidence about human behavior. Its purpose here is narrower and more modest than the original plan intended: it gives every automated run something concrete to be compared against on purely structural terms – how many

experiments were run, whether the search broadened or narrowed, when it stopped – without claiming anything about what a real person would have done.

One consequence of planning ahead for this step shaped everything that follows regardless: the six-item action vocabulary meant for the reference trace was adopted as the logging format for every automated run as well, so comparing the two required no change to the rest of the pipeline once the reference trace existed.

3.2 Step 2: turning the instruction file into a configuration

The first executed step reduces the free-text instruction file to a small, structured set of choices, the same way a hyperparameter vector reduces an algorithm to something whose behavior can be studied systematically rather than described anecdotally. Five instructional choices were identified by reading through example instruction files and noting which sentences plausibly influence behavior; each choice was restricted to exactly two levels, giving a five-bit configuration.

The metric axis controls how precisely the instructions define success: at one level the file simply says to "evaluate model quality in whatever way you judge appropriate"; at the other it specifies exactly what to compute – mean five-fold cross-validated accuracy together with its standard deviation across folds, with the standard deviation used only to break close ties. The breadth axis controls whether the agent must commit to a single model family, Random Forest or Gradient Boosting, for an entire run, or is free to try both. The stopping-rule axis controls the experiment budget: either a fixed budget of exactly three experiments, or an adaptive rule that allows up to eight experiments but stops as soon as one experiment fails to improve on the best result so far by more than a small threshold. The output-format axis controls whether responses must be terse and numeric, or must include full, reasoned explanations of hypotheses and interpretations. The emphasis axis controls search strategy: whether the agent should prioritize trying several different configurations before refining any one of them, or should immediately refine the first promising result it finds with nearby variations.

A configuration is then simply a vector of five zeros and ones, for instance (1, 0, 0, 1, 1), one bit per axis in the order metric, breadth, stopping rule, output format, emphasis. Rather than writing out each of the 32 possible instruction files by hand, a single template with a placeholder for each axis is filled in automatically from a configuration vector by a short script. This guarantees that any two generated instruction files differ only in the axes that were actually varied between them – a property the later steps depend on, since attributing a behavioral difference to a specific axis only makes sense if nothing else changed at the same time.

3.3 Step 3: choosing which configurations to run, and running them

With five two-level axes, testing every combination means 32 configurations, and because a research loop is not perfectly deterministic, each configuration would ideally be repeated several times to separate a genuine effect of the configuration from ordinary run-to-run variation. Running all 32 configurations with several repeats each was outside the scope of what a single pilot could cover, so an eight-configuration Plackett–Burman design was used instead: a specific, non-arbitrary subset of the 32 possible configurations, generated by the classical cyclic construction (a base pattern of +, +, +, −, +, −, − cyclically shifted seven times, plus one row of all minus signs), chosen so that the individual effect of every one of the five axes can still be estimated without being distorted by any of the others. The price of using only eight runs is that interactions between pairs of axes – for instance, whether the effect of the emphasis axis depends on the breadth axis – cannot be told apart from each other; this is a standard and explicitly accepted trade-off, and Section 5 shows a concrete case where it matters. Table 1 lists the resulting eight configurations.

Each configuration was run twice, with two different random seeds, giving sixteen runs in total. A larger study would use three to five repeats per configuration to separate genuine configuration effects from

Table 1: The eight configurations used in the ablation study. Columns are the five axes from Section 3.2 in order: metric, breadth, stopping rule, output format, emphasis.

Configuration	Metric	Breadth	Stopping	Output	Emphasis
0	1	1	1	0	1
1	0	1	1	1	0
2	0	0	1	1	1
3	1	0	0	1	1
4	0	1	0	0	1
5	1	0	1	0	0
6	1	1	0	1	0
7	0	0	0	0	0

noise more reliably; two were used here to keep the pilot within reach of a single sitting, and this is listed plainly as a limitation in Section 7.

Running a configuration means generating its instruction file and then executing the loop it describes against a real machine learning task. Every run addresses the same question – how does hyperparameter choice affect the generalization accuracy of Random Forest and Gradient Boosting classifiers on the Breast Cancer Wisconsin diagnostic dataset, a small, standard benchmark bundled with scikit-learn – using the same fixed 70/30 split of the data into a training and a validation portion, so that only the instructions and the loop’s own randomness differ from one run to the next. At each step of a run, the agent can request one experiment by naming a model family and a set of hyperparameters; an ordinary piece of Python code, not a language model, then actually fits that model, computes five-fold cross-validated accuracy and its standard deviation on the training portion, and computes accuracy on the held-out validation portion, and reports these real numbers back. This executor is the one part of the loop that never involves a language model: every number the agent sees is a genuine measurement from an actually-trained scikit-learn model. The agent, in turn, responds at each step with one of two structured actions: propose an experiment with a stated hypothesis, or interpret the last result and decide whether to continue or stop, giving a final recommendation if stopping. Logging every action this way, rather than as free-flowing prose, is what makes the next step mechanical rather than a difficult natural-language-understanding problem.

3.3.1 What `program.md` actually contains

Most of `program.md` is boilerplate that never changes between configurations: a fixed statement of the research question, a fixed description of the dataset and the tool available for running an experiment, and a fixed specification of the two-shape JSON format every response must follow (shown once, in full, in Listing 1, since it is identical across all 32 possible configurations). Only four short paragraphs – evaluation, search strategy, budget and stopping, reporting style – actually change, one per axis (the fifth axis, emphasis, is folded into the search-strategy paragraph alongside breadth). Listing 2 shows these four paragraphs exactly as generated for two configurations at opposite corners of the design: configuration 7, $(M, B, S, O, E) = (0, 0, 0, 0, 0)$ – vague metric, narrow, fixed budget, terse, explore-first – and configuration 1, $(0, 1, 1, 1, 0)$ – vague metric, broad, adaptive, verbose, explore-first. Everything outside these four paragraphs is character-for-character identical between the two files.

Listing 1: The response format specification. Identical in every one of the 32 possible `program.md` files; shown once here rather than in every listing below.

```
At every step, respond with a single JSON object and nothing else,
matching one of these two shapes:

Proposing an experiment:
{"action": "propose", "hypothesis": "<why you think this configuration
```

```
will help>", "model": "random_forest" | "gradient_boosting",
"params": {"<hyperparameter>": <value>, ...}}
```

```
Interpreting the last result and deciding what happens next:
{"action": "interpret", "interpretation": "<what the result tells you>",
"decision": "continue" | "stop", "final_recommendation": "<only
required when decision == stop: your final recommended model and
hyperparameters, and why>"}
```

Listing 2: The four instructional paragraphs that actually vary, for configuration 7 (top, all axes at level 0) and configuration 1 (bottom, $(M, B, S, O, E) = (0, 1, 1, 1, 0)$). This is the entire textual difference between the two rendered instruction files.

```
--- Configuration 7: (M,B,S,O,E) = (0,0,0,0,0) ---

## Evaluation
Evaluate model quality in whatever way you judge appropriate.

## Search strategy
Stay within a single model family per experiment. Pick either
RandomForestClassifier or GradientBoostingClassifier for the whole
session unless you have exhausted reasonable hyperparameters for it
and explicitly decide to switch.

Prioritize breadth of search: try several meaningfully different
configurations before you spend more than one experiment refining
any single one of them.

## Budget and stopping
You have a fixed budget of exactly 3 experiments in total. Run
exactly 3, then stop and report your final recommendation,
regardless of the outcome of experiment 3.

## Reporting style
Keep every response short: state the action you are taking and the
numbers involved, with minimal prose.

--- Configuration 1: (M,B,S,O,E) = (0,1,1,1,0) ---

## Evaluation
Evaluate model quality in whatever way you judge appropriate.

## Search strategy
You are free to compare RandomForestClassifier and
GradientBoostingClassifier side by side within the same session, and
to vary multiple hyperparameters of each at once.

Prioritize breadth of search: try several meaningfully different
configurations before you spend more than one experiment refining
any single one of them.

## Budget and stopping
You have a budget of up to 8 experiments. After each experiment,
decide whether to continue: stop as soon as an experiment fails to
improve the evaluation metric (defined above) by more than 0.002
over the best result so far, or when you reach 8 experiments,
whichever comes first.

## Reporting style
Explain your reasoning at every step in full sentences: what you
hypothesize, why, what you expect to observe, and how you interpret
the actual result relative to that expectation.
```

3.3.2 What a run actually looks like

To make the loop mechanics concrete, Listing 3 reproduces the first two iterations of one real run, generated from configuration 1 above. Every field under `execute` is the executor’s real output for the stated model and hyperparameters – not paraphrased or rounded by hand. The agent proposes Random

Forest with 100 trees and depth 5 as a first, unremarkable baseline; the executor reports back a mean cross-validated accuracy of 0.962. Because configuration 1’s search-strategy paragraph favors breadth before refinement, the second proposal switches model family entirely, to Gradient Boosting, rather than tuning the Random Forest further; that experiment comes back weaker (0.945 vs. 0.962), and because configuration 1’s stopping rule requires an improvement of more than 0.002 to continue, this single disappointing exploratory step is itself what ends the run after only two experiments – an interaction between the breadth and stopping-rule paragraphs that is typical of the kind of behavior this study tries to measure systematically rather than notice anecdotally.

Listing 3: First two iterations of a real run under configuration 1. Fields under `execute` are genuine executor output.

```

step 1 propose model=random_forest, params={n_estimators: 100,
      max_depth: 5}
      "I'll start with a Random Forest of 100 trees and moderate
      depth 5, a sensible default for a small tabular dataset,
      to establish a baseline before exploring further."
step 1 execute cv_accuracy_mean=0.9623 cv_accuracy_std=0.0113
      val_accuracy=0.9474
step 1 interpret "The baseline Random Forest already reaches a
      cross-validated accuracy of about 0.962 ... this becomes
      the reference to beat."
step 1 decide continue

step 2 propose model=gradient_boosting, params={n_estimators:100,
      max_depth: 3, learning_rate: 0.1}
      "Since I am encouraged to explore breadth before refining,
      I will now try a completely different model family ..."
step 2 execute cv_accuracy_mean=0.9448 cv_accuracy_std=0.0150
      val_accuracy=0.9357
step 2 interpret "Gradient Boosting underperforms the Random
      Forest baseline here (0.945 vs 0.962) ... well past the
      point of counting as an improvement."
step 2 decide stop
      final_recommendation: "random_forest, n_estimators=100,
      max_depth=5 (cv_mean~0.962). ... the exploratory experiment
      itself triggered the stopping rule."

```

3.4 Step 4: turning a run into a handful of numbers

Because each run already produces a structured, itemized record rather than a wall of prose, reducing it to a row of numbers is a matter of straightforward parsing rather than interpretation. The harder question is not how to compute a number from a run, but which numbers are worth computing in the first place. Six were chosen, and each earns its place by answering a specific question about the run that the raw transcript does not answer on its own – not because it is the only way that question could be measured, but because it is a simple, cheaply computed answer to a question that matters for this study specifically. They fall into three groups.

The first group asks how much research actually happened, and how widely it ranged, independent of whether any of it paid off. **Number of experiments** is the most basic possible answer to “how much research happened”: every experiment corresponds to one real model fit, so this is a direct count of executed work, not a proxy for it. **Number of distinct model families tried** (one or two, depending on whether the agent ever switched between Random Forest and Gradient Boosting) matters because the breadth axis (Section 3.2) only *permits* switching families – it does not force it – so this quantity is the only way to check whether that permission was actually exercised, as opposed to merely granted. **Search width** (the range between the largest and smallest tree-count value tried within a run) answers the same kind of question one level down, inside a single model family: even a run that never switches models can still search a narrow or a wide slice of that model’s hyperparameters, and this is the simplest available measurement of which one happened.

The second group asks whether any of that activity was actually worth doing. **Best cross-validated**

accuracy reached is the one quantity in this study that measures the outcome of the research rather than the shape of the process that produced it – the “product,” in the introduction’s terms, as opposed to the process. It matters specifically because process and outcome can come apart: an instruction file could change how an agent searches without changing how well it eventually does, or vice versa, and only measuring one of the two would miss half of that picture. It is also the quantity used for the causal-validation check in Step 6 (Section 3.6), since “how good is the final answer” is the outcome most people actually care about when they read a real research report, more than any process detail.

The third group is not really about the agent at all, but about whether this measurement pipeline can be trusted. **Average length of the agent’s stated hypotheses, in characters**, is a deliberately crude proxy for verbosity, and it is included for a specific reason: the output-format axis (Section 3.2) directly and unambiguously instructs the agent to be either terse or fully reasoned, so this is the one quantity in the whole study where the “correct” answer is known in advance. If the fitted model in Step 5 fails to identify the output-format axis as the dominant driver of this particular quantity, that is a sign that something in the pipeline – the template, the logging, the parsing, the model fit – is broken, not that the agent is behaving unexpectedly. It functions as a calibration instrument for every other result in this report, which is also why it is discussed first in Section 5. Finally, **distance to the Step 1 reference trace** is the one quantity that looks at a run’s overall shape rather than any single number extracted from it, and is the closest operational answer this pilot has to the original agenda’s actual motivating question: not “how many experiments did the agent run” in isolation, but “does the overall rhythm of this research process resemble how a research process is normally carried out.” Its meaning is necessarily weaker than intended, for the reasons given in Section 3.1, but the question it is trying to answer is the same one that motivated collecting a reference trace at all.

That last quantity is computed as follows. Every run’s sequence of logged actions is collapsed to a four-symbol alphabet (propose, execute, interpret, decide), merging the finer-grained categories used in the reference trace (hypothesize and design-an-experiment both collapse to propose; run-it and observe-the-result both collapse to execute) and merging any immediately repeated symbol into one. The resulting sequence is compared to the reference trace’s own sequence using edit distance – the smallest number of insertions, deletions, or substitutions needed to turn one sequence into the other – normalized by the length of the longer of the two sequences, giving a score between 0 (identical shape) and 1 (maximally different). This is a coarse measure: it looks only at the shape of a run (how many steps of each kind, in what order) and ignores their content entirely. Still, it captures something the other five quantities do not, namely whether a run’s overall research rhythm – how many times it circles through propose-execute-interpret before deciding to stop – resembles the reference session’s rhythm or not. The result of this step is a single table with one row per run: the five-bit configuration, which repeat it was, and these six numbers.

3.5 Step 5: a simple model of configuration and behavior

For each of the six behavioral quantities y , a linear model of the form

$$y = \beta_0 + \beta_M x_M + \beta_B x_B + \beta_S x_S + \beta_O x_O + \beta_E x_E + \varepsilon$$

was fit by ordinary least squares, where each $x_i \in \{-1, +1\}$ codes one of the five axes (level 0 as -1 , level 1 as $+1$) and ε is the usual error term. Before fitting, the two repeats of each configuration were averaged together, so the model is fit on $n = 8$ configuration-level values rather than sixteen individual runs – one row per row of Table 1 – matching the resolution the ablation design was actually built to support.

Coding the axes as ± 1 , together with the fact that the Plackett–Burman design is orthogonal (the coded columns for any two axes are uncorrelated across the eight configurations), has two useful consequences. First, each β_i can be estimated independently of the others: the fitted value of β_M , say, does not depend on which other axes happened to be varied alongside it. Second, the share of the model’s total explained

variance attributable to axis i is simply $\beta_i^2 / \sum_j \beta_j^2$, giving a direct, additive breakdown of "how much of the variation in y this axis accounts for" without any further computation.

For every coefficient, `statsmodels` additionally reports a standard error, a t -statistic, and a two-sided p -value for the null hypothesis $\beta_i = 0$ (that this axis has no effect on y) against a t -distribution. These are included in full in Section 5 rather than only summarized, together with one number that has to be kept in view when reading them: with $n = 8$ data points and six free parameters (the intercept plus five slopes), the model has only $n - 6 = 2$ residual degrees of freedom. A t -test with two degrees of freedom is a real, valid test, but it is not a well-powered one – it can only distinguish an effect from noise when the effect is large relative to how much the eight configurations happen to vary, and its p -values should be read as a rough indicator of which axes stand out sharply from the rest, not as evidence with the reliability a larger study would provide. This is stated plainly here rather than left implicit, because a very small p -value from a two-degrees-of-freedom test looks the same on the page as one from a much larger, better-powered study, and only is not.

As a non-parametric cross-check, a shallow decision tree (limited to three levels) was also fit for each behavioral quantity, to confirm it splits on the same axes the linear model considers important; this does not depend on the small-sample caveat above, since it makes no distributional assumptions, but with only eight training points it should likewise be read as a consistency check rather than as an independently reliable model.

3.6 Step 6: checking the model with a real intervention

A model that only fits the data used to build it has not yet demonstrated anything beyond correlation. The last step is meant to turn the fitted model into a claim that can actually be tested: one configuration, $(0, 0, 0, 0, 1)$, was chosen because it was not among the eight used to fit the model in Step 5. Using the fitted model, two of its behavioral quantities were predicted *before* this configuration was ever run. The corresponding instruction file was then generated and executed twice, using two random seeds not used anywhere else in this pilot, and the predictions were compared against what actually happened. This is the difference between saying "the model fits the data it was trained on" and saying "the model's implied claim – if you set the instructions this way, you should get approximately this behavior – held up when we actually tried it."

4 Experimental Setup

Every run used the Breast Cancer Wisconsin diagnostic dataset as bundled with `scikit-learn`: 569 samples, 30 numeric features, a binary classification target. A single 70/30 train/validation split, fixed with the same random seed across every run, ensured that only the instructions and the loop's own stochastic behavior varied between configurations. The two model families available to the agent were `scikit-learn`'s `RandomForestClassifier` and `GradientBoostingClassifier`; for the former the agent could vary the number of trees, tree depth, minimum leaf size, and the fraction of features considered at each split, and for the latter the number of boosting stages, tree depth, learning rate, and subsample fraction. Regardless of what a given configuration's instructions emphasized, the executor always reported the same measurements back: five-fold cross-validated accuracy with its standard deviation on the training split, and accuracy on the held-out validation split.

In total, the pilot consisted of eight ablation configurations run twice each, plus one held-out configuration also run twice, for eighteen runs and forty-six individual model-fitting experiments across all of them combined. All code, generated instruction files, and raw run records are kept alongside this report so the pipeline can be re-run or inspected directly.

5 Results

Table 2 summarizes, for each of the six behavioral quantities from Step 4, which axis the fitted linear model identifies as the dominant driver, how large a share of the variation it accounts for, the two-sided p -value for that axis’s coefficient (recall: from a two-degrees-of-freedom test, per Section 3.5), and the model’s overall fit (R^2) across the eight configuration-level values. Full coefficient tables, with standard errors, for the two quantities discussed in most depth below follow in Tables 4 and 3.

Table 2: Which axis explains which behavioral quantity, according to the fitted linear model. p is for the dominant axis’s coefficient.

Behavioral quantity	Dominant axis	Var. share	p	Direction of effect	R^2
Verbosity (avg. proposal length)	Output format	74%	0.027	full explanations $\rightarrow$ roughly 2–4 $\times$ longer proposals	0.96
Model families tried	Breadth, output, emphasis (tied)	33% each	$< 10^{-29}$	broad $\uparrow$, verbose $\uparrow$, exploit-first $\downarrow$	1.00
Best accuracy reached	Emphasis	84%	2×10^{-6}	exploit-first $\rightarrow$ +0.0025 mean accuracy	1.00
Number of experiments	Stopping, emphasis (tied)	50% each	0.29	adaptive stopping $\downarrow$, exploit-first $\uparrow$	0.67
Search width	Emphasis	49%	0.16	exploit-first $\rightarrow$ wider spread	0.83
Distance to reference trace	Stopping, emphasis (tied)	50% each	0.29	adaptive stopping $\uparrow$ distance, exploit-first $\downarrow$ distance	0.67

The first row is, by design, close to a sanity check. The output-format axis directly instructs the agent to write either terse or fully reasoned responses, so finding that it explains 74% of the variation in how long the agent’s stated hypotheses are is exactly what should happen if the whole pipeline – from generating the instruction file, through running the loop, logging its steps, parsing them into numbers, and fitting a model – is working correctly end to end. Table 3 gives the full picture behind that 74%: all five coefficients, their standard errors, and their p -values. Only the output-format axis reaches conventional significance ($p = 0.027$); the other four axes have p -values between 0.14 and 0.54 and correspondingly small variance shares, consistent with genuinely having little to do with how long the agent’s responses are. If some unrelated axis had turned out to dominate verbosity instead, that would have pointed to a bug somewhere in the pipeline rather than a genuine finding. Because this check passes cleanly, it is reasonable to give some credence to the less obvious result that follows.

Table 3: Full linear-model coefficients for verbosity (average proposal length, in characters). Coefficient is on the ± 1 -coded axis; a level-0-to-1 change is twice this value. $n = 8$, $\text{df}_{\text{resid}} = 2$.

Axis	Coefficient	Std. error	t	p
Metric (M)	−4.16	5.65	−0.74	0.538
Breadth (B)	+11.56	5.65	2.05	0.177
Stopping (S)	+13.27	5.65	2.35	0.143
Output format (O)	+33.44	5.65	5.92	0.027
Emphasis (E)	−8.47	5.65	−1.50	0.272

That result concerns the emphasis axis, which says nothing about accuracy at all – it only tells the agent whether to explore several configurations before refining any one of them, or to refine the first promising

result immediately. Despite that, it accounts for 84% of the variation in the best accuracy reached across the eight configurations, with configurations that favor immediate refinement outperforming those that favor broad exploration first by about a quarter of an accuracy percentage point on average. Table 4 gives the full coefficients. Four of the five p -values here are extremely small, down to 2×10^{-6} for the emphasis axis – but this table is exactly the case Section 3.5 warned about: with only $n = 8$ points and two residual degrees of freedom, the achieved accuracy across these eight configurations happens to vary so little and so consistently with a linear combination of the axes ($R^2 > 0.9999$) that almost any nonzero coefficient comes out looking significant by this test. The p -values are real numbers, correctly computed, and it is still true that emphasis stands out from the other four axes by roughly an order of magnitude in both effect size and variance share – but they should not be read with the confidence a t -test at $n = 8$ would carry in, say, a clinical trial. The effect is a small one in absolute terms (a quarter of an accuracy percentage point) that is consistent and clearly dominant *within this design*, which is a more modest and more honest claim than “statistically significant at $p < 10^{-5}$ ” makes it sound.

Table 4: Full linear-model coefficients for best cross-validated accuracy reached. Coefficient is on the ± 1 -coded axis; a level-0-to-1 change is twice this value. $n = 8$, $\text{df}_{\text{resid}} = 2$. See the surrounding text for why these p -values should be read cautiously despite their small size.

Axis	Coefficient	Std. error	t	p
Metric (M)	−0.000314	0.0000019	−168	3.6×10^{-5}
Breadth (B)	+0.000002	0.0000019	1.0	0.423
Stopping (S)	−0.000314	0.0000019	−168	3.6×10^{-5}
Output format (O)	−0.000314	0.0000019	−168	3.6×10^{-5}
Emphasis (E)	+0.001252	0.0000019	668	2.2×10^{-6}

There is a plausible mechanism behind the emphasis effect, small and imprecisely bounded as it is: with only two to four experiments available in most of these runs, a strategy that commits early to refining a promising region of the hyperparameter space has less room to waste an experiment exploring a probably-worse region than a strategy that insists on breadth first. It is exactly the kind of effect that would be invisible without decomposing the instruction file into separate axes and measuring each one’s contribution directly – one would otherwise only observe that some instruction files happen to lead to better results than others, without being able to say why.

The remaining rows are more mechanical but still informative. The breadth axis, unsurprisingly, is one of three tied drivers of how many distinct model families get tried, since it is the axis that permits or forbids trying more than one family in the first place; that the output-format and emphasis axes tie with it suggests that being asked to reason at length, or to explore broadly, also nudges the agent toward trying both families rather than one, even though neither axis mentions model families directly. The number of experiments run is split evenly between the stopping-rule axis, which sets the budget in the first place, and the emphasis axis, which affects how quickly that budget gets used up by determining whether early experiments tend to keep improving or not.

The last row, distance to the reference trace from Section 3.1, is explained by exactly the same two axes and in almost exactly the same proportion as the number of experiments – which makes sense, since the reference trace itself ran seven experiments and the distance measure is built from the same coarse propose-execute-interpret-decide rhythm that the experiment count summarizes. Runs under an adaptive stopping rule end up further from the reference trace on average, and runs that favor immediate refinement end up closer to it. Given that the reference trace was authored rather than recorded (Section 3.1), this result should be read narrowly: it shows that the process-similarity measure behaves sensibly and picks up a real structural pattern, not that immediate-refinement instructions make an agent behave more like an actual human researcher.

The held-out configuration used for causal validation – narrow model family, an unspecified evaluation

metric, a fixed budget of exactly three experiments, terse output, and immediate-refinement emphasis – was deliberately excluded from the eight configurations used to fit the model above. Table 5 compares what the fitted model predicted for two of its behavioral quantities, before this configuration was ever executed, against what was actually observed once it was run twice with two fresh random seeds.

Table 5: Predicted versus observed behavior for the held-out configuration.

Behavioral quantity	Predicted (before running)	Observed (after running)	Difference
Number of experiments	3.25	3.00	−0.25
Best accuracy reached	0.9648	0.9610	−0.0038

The number-of-experiments prediction is essentially exact: the model correctly identified that this configuration’s fixed budget of three experiments would dominate, and predicted a value within a quarter of an experiment of what actually happened. The accuracy prediction is off by about 0.004, in the direction of overestimating how much the immediate-refinement strategy helps. This is not a random miss; it has a specific explanation. Because the eight-configuration design used in Step 5 cannot separate interactions between axes, the estimated benefit of immediate refinement was necessarily averaged across configurations that also varied the breadth and stopping-rule axes. If the true benefit of immediate refinement actually depends on, say, whether the agent is also allowed to compare two model families, then a model that treats it as a fixed, additive bonus will get some configurations slightly wrong – which is exactly what happened here. Read this way, the miss is not a failure of the method; a wrong-but-explainable prediction that points at a specific, checkable gap (an interaction the design could not see) is a more useful outcome than either a prediction that always happens to be right, or a validation step that is skipped altogether.

6 How much of the black box has actually been opened

The introduction described the problem this project set out to address as a black box: change `program.md` and the agent’s behavior changes, but there was no way to say which part of the instructions was responsible for which part of the change, nor to predict what a change not yet tried would do. It is worth stating precisely what “opening” that black box meant in practice, what evidence from Sections 3 and 5 actually supports that claim, and – just as importantly – what kind of black box this project did *not* open, so that the white-box claim in the title of this line of work is not read as broader than it is.

What became visible. Three specific things changed between “we observe that the instructions matter” and where this pilot ends up. First, the input stopped being an undifferentiated block of text and became a small, labeled vector: five named axes, each with a stated meaning, rather than one opaque file. This alone is a real step – it is the difference between saying “the instructions” and saying “the stopping-rule paragraph, specifically” – and it is what made everything downstream possible. Second, the relationship between that vector and the agent’s behavior was captured in a model whose insides can be read directly: a linear equation with five coefficients, not a black-box predictor of its own. Every claim in Table 2 can be traced back to one number in one regression, and Tables 3 and 4 show all five of those numbers, not just the largest one. Third, and most importantly, one specific claim generated by that model was checked against a real intervention before the outcome was known (Section 3.6), rather than only checked against the data used to build the model. A model that merely correlates with data it was fit on can always be produced; a model whose stated prediction is checked against a run that had not happened yet, and is found to be right for one quantity and specifically, explainably wrong for another, has done something a black box by definition cannot do: made a falsifiable claim and been tested on it.

What stayed exactly as opaque as before. None of this says anything about *why* a large language model responds to instructions the way it does at the level of what actually happens inside it – there is no claim here about attention patterns, learned representations, or any mechanism internal to the model. The language model itself remains as much of a black box after this report as before it; what changed is that its *external, input-output behavior*, for one narrow task and one small set of instructional choices, now has a description that is interpretable and testable, in the same sense that a linear regression relating a drug’s dosage to a patient outcome is interpretable without anyone needing to understand human biochemistry to use it. This is precisely the sense in which AutoML hyperparameter-importance tools “open” the black box of a learning algorithm (Section 2): functional ANOVA over a Random Forest’s hyperparameters tells you that tree depth matters more than leaf size for a given dataset without telling you anything about how decision trees or bagging actually work internally. The white box here is a white box at the level of *behavior*, not at the level of *mechanism*, and the two should not be confused.

Where the white-box claim is, and is not, entitled to travel. Even restricted to behavior, the claim earned by this pilot is narrower than it might sound. It is entitled to say: for this task, this dataset, these five axes, and this specific stand-in for an agent, the relationship between instruction and behavior can be measured, summarized in an interpretable model, and used to make a checkable prediction that at least partially held up. It is not entitled to say that this relationship holds for other tasks, other datasets, a larger or differently chosen set of axes, or – most importantly, restating Section 7’s central point – for an actual independent language model rather than for the author acting out that role. Every one of the specific numbers in Section 5 (the 74%, the 84%, the p -values) describes this pilot’s own small, self-contained system, not autoresearch loops in general. What generalizes, if anything does, is the method: the demonstration that an instruction file can be treated as a configuration space, ablated rather than randomly sampled, modeled with something interpretable, and checked with a real intervention, at all. Whether the specific findings generalize is exactly the open, unanswered question the next steps in Section 8 are aimed at.

To put the whole claim in one sentence: this project turned a black box into a white box in the narrow, specific sense of turning “the instructions matter, somehow” into “here is a model of which instruction changes which behavior, by how much, and it made one falsifiable prediction that was checked” – and it did this for one small pilot system standing in for the real thing, not for autoresearch loops as a general phenomenon.

7 Limitations

The most significant limitation of this pilot concerns how the agent’s decisions were actually produced. Genuinely independent behavior – decisions made by a language model that has seen nothing but the instruction file for that specific run, with no memory of any other run – requires a way to make many separate, freshly-started calls to such a model. No such access was available while this pilot was carried out. In its place, the author personally played the role of the agent for every one of the eighteen runs, reading each generated instruction file and trying to act on only what it said. This is a genuine confound: the author already knew, throughout, that this was a controlled study and roughly what each axis was meant to test, which a language model given only the instruction file would not know. The results in Section 5 should therefore be read as a demonstration that the analysis pipeline itself works correctly and can surface plausible, self-consistent effects, not as an established fact about how an independent language model behaves under these instructions. Replacing this substitution with real, independent calls to a language model, and checking whether the same effects reappear, is the single most important next step for this work.

A second limitation, closely related to the first, is that the reference trace used in Section 5 to compute process-similarity is authored, not recorded, for exactly the reason given in Section 3.1: no person was actually observed solving the reference problem. The distance-to-reference-trace numbers reported

above should therefore be read as evidence that the measurement pipeline works and picks up a real structural pattern, not as evidence about how closely an agent’s behavior resembles genuine human research behavior. Replacing the authored trace with an actual recorded session – the original plan for this step – remains open, and the logging format used throughout was chosen precisely so that doing so later will not require re-running anything already done.

A third limitation is scale. Two repeats per configuration were used rather than the three to five that would more reliably separate a genuine configuration effect from ordinary noise, and eight configurations support only individual-axis effects, not interactions between pairs of axes – a limitation the causal-validation result above shows is not merely theoretical. A fourth and final limitation is scope: every experiment concerns one small dataset and two model families with a modest set of hyperparameters, so the specific numbers reported here should not be assumed to generalize beyond this setting without repeating the study elsewhere.

8 Conclusion

This pilot carried out all six planned steps toward explaining, rather than merely observing, how an instruction file shapes the behavior of an automated research loop, by treating the instruction file the same way automated machine learning treats a hyperparameter vector: as a small set of discrete choices whose individual effect on behavior can be measured with a fractional factorial design, summarized with a simple interpretable model, checked with a real held-out intervention, and related to a reference research trace. Every number in this report came from actually executed machine learning experiments; two of the six steps, however, stand in for something that was meant to be independent of the author and were not – the agent’s decisions in every run, and the reference trace they are compared against – and the results should be weighed with that firmly in mind.

The method recovered an expected relationship cleanly (the instruction to write full explanations is the dominant driver of how much text the agent produces) and surfaced a less obvious one (whether the agent is told to explore broadly or refine immediately turns out to be the strongest single predictor of the accuracy it eventually reaches). Its held-out validation step produced a genuinely informative result: one quantity predicted almost exactly, and one prediction that missed by a small amount in a way that points precisely at what the design could not see – an interaction between the emphasis axis and one of the others. The process-similarity measure built from the reference trace behaved sensibly, picking up the same stopping-rule and emphasis effects as the raw experiment count, though it cannot yet say anything about actual human research behavior.

The most useful next steps, in order, are: replacing the author’s own role as the acting agent with independent calls to a language model, to check whether the effects found here survive that change; replacing the authored reference trace with an actual recorded human session, which is the one substitution that cannot be worked around computationally; expanding the ablation design so that interactions between axes, in particular those involving the emphasis axis, can be estimated directly rather than only inferred from a validation miss; and repeating the study on more than one dataset to see how much of what was found here is specific to this particular benchmark.

References

- [1] C. Fawcett and H. H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4):431–458, 2016.
- [2] F. Hutter, H. Hoos, and K. Leyton-Brown. An efficient approach for assessing hyperparameter importance. *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014.

- [3] R. L. Plackett and J. P. Burman. The design of optimum multifactorial experiments. *Biometrika*, 33(4):305–325, 1946.